What is C++?

C++ is a cross-platform language that can be used to create high-performan e applications.

C++ was developed by Bjarne Stroustrup, as an extension to the C language.

C++ gives programmers a high level of control over system resources and memory.

The language was updated 3 major times in 2011, 2014, and 2017 to C++11, C++14, and C++17.

Why Use C++

C++ is one of the world's most popular programming languages.

C++ can be fo und in today's operating systems, Graphical User Interfaces, and embedded systems.

C++ is an object -oriented programming language which gives a clear structure to programs and allows code to be reused, lowering development costs.

C++ is portable and can be used to develop applications that can be adapted to multiple platforms.

C++ is fun and easy to learn!

As C++ is close to C#, and Jav a, it ma kes it easy for programmers to switch to C++ or vice versa

---

Get Started

This tutorial will teach you the basics of C++.

It is not necessary to have any prior programming experience.

C++ Get Started

To start using C++, you need two things:

- A text editor, like Notepad, to write C++ code
- A compiler, like GCC, to translate the C++ code into a language that the computer will understand

There are many text editors and compilers to choose from. In this tutorial, we will use an IDE (see below).

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

C++ Install IDE

An IDE (Integrated Development Environment) is used to edit AND compile the code.

Popular IDE's include Code::Blocks, Eclipse, and Visual Studio. These are all free, and they can be used to both edit and debug C++ code.

Note: Web-based IDE's can work as well, but functionality is limited.

We will use **Code::Blocks** in our tutorial, which we think is a good place to start.

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5

You can find the latest version of Codeblocks at

Download the `mingw-setup.exe` file, which will install the text editor with a compiler.

C++ Quickstart

Let's create our first C++ file.

Open Codeblocks and go to **File > New > Empty File**

Write the following C++ code and save the file as `myfirstprogram.cpp` (**File > Save File as**)

myfirstprogram.cpp

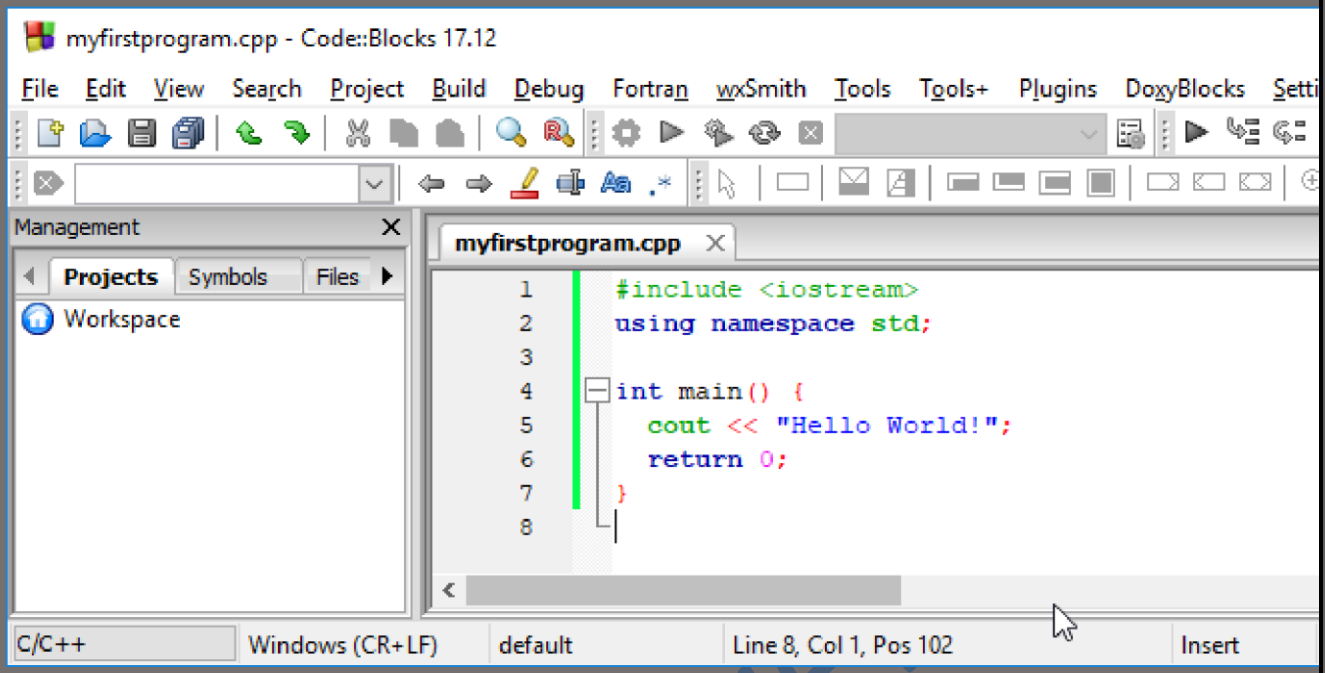
```
#include <iostream>
using namespace std;

int main() {
  cout <<"Hello World!"
  return0;
}
```

Don't worry if you don't understand the code above - we will discuss it in detail in later chapters. For now, focus on how to run the code.

In Codeblocks, it should look like this:

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM



Then, go to **Build > Build and Run** (execute) the program. The result will look something to this:

```
Hello World!
Process returned 0 (0x0) execution time : 0.011 s
Press any key to continue.
```

Congratulations! You have now written and executed your first C++ program.

When learning C++ at W3Schools.com, you can use our "Run Example" tool, which shows both the code and the result. This will make it easier for you to understand every part as we move forward:

myfirstprogram.cpp

Code:

```
#include
<iostream> using
namespace std;
int main() {
cout << "Hello
World!";
return 0; }
```

Result:

Click: https://chat.whatsapp.com/GeeIrau2Nit0D4F5DjDiGM

```
Hello World!
```

C++ Syntax

Let's break up the following code to understand it better:

Example

```
#include <iostream>
using namespace std;

int main() {
  cout <<"Hello World;"
  return0;
}
```

Run example »

Example ex

Line

#inclu

objects, such <iostream> is a **header file library** that lets us work with input and output cout (used in line 5). Header files add functionality to C++ programs.

Line 2:

standard library. namespace std means that we can use names for objects and variables from the

ou don't understand how #include <iostream> and using namespace std works. s something that (almost) always appears in your program.

Line 3:

line. C++ ignores white space.

Line 4: a **function**

r thing that always appear in a C++ program, is int main(). This is called y code inside its curly brackets {} will be executed.

Line 5: cout (pronounced "see-out") is an **object** used together with the *insertion operator* (<<) output/print tex our example it will output "Hello World".

Note: + statement ends with a semicolon ;.

Note: The bod int main() could also been written as:

of

```
int main () { cout << "Hello World! "; return 0; }
```

Remember: The compiler ignores white spaces. However, multiple lines makes the code more readable.

Line 6: return 0 ends the main function.

Line 7: Do not forget to add the closing curly bracket } to actually end the main function.

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Omitting Namespace

You might see some C++ programs that runs without the standard namespace library. The `usin` line can be omitted and replaced with the `std` keyword, followed by `namespace std` for some objects:

the `::`

ample

```
clude <iostrea

ain() { cout   "Hello World!"
  <<  0;
```

int td::
eturn

tput (Print Text)

The `out` t, together with the `<<` operator, is used to output values/print text:

ample

```
ude <iostream
 namespace st

ain() {
 <<          llo World!"
      0;
```

int
out
eturn
}

example »

Run

n add as many at t
tput:

`cout` objects as you want. However, note that it does not insert a ne line

ample

```
#include <iostream>
using namespace std;
 int main() {   cout
<< "Hello World!";
cout << "I am
learning C++";
return 0;
}
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

C++ Comments

Comments can be used to explain C++ code, and to make it more readable. It can also be used to prevent execution when testing alternative code. Comments can be singled-lined or multilined.

Single -line comments start with two forward slashes (`//`).

Any text between `//` and the end of the line is ignored by the compiler (will not be executed).

This example uses a single -line comment before a line of code:

Example

```
// This is a comment
cout <<"Hello World;"
```

Run example »

This example uses a single -line comment at the end of a line of code:

Example

```
cout <<"Hello World;"// This is a comment
```

Run example »

C++ Multi-line Comments

Multi-line comments start with `/*` and ends with `*/`.

Any text between `/*` and `*/` will be ignored by the compiler:

Example

```
/* The code below will print the words Hello World!
to the screen, and it is amazing */
cout <<"Hello World;"
```

Run example »

C++ Variables

Variables are containers for storing data values.

In C++, there are different **types** of variables (defined with different keywords), for example:

- `int` - stores integers (whole numbers), without decimals, such as 123 or -123

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

- `double` - stores floating point numbers, with decimals, such as 19.99 or -19.99
- `char` - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- `string` - stores text, such as "Hello World". String values are surrounded by double quotes
- `bool` - stores values with two states: true or false

Declaring (Creating) Variables

To create a variable, you must specify the type and assign it a value:

Syntax

type variable = *value*;

Where *type* is one of C++ types (such as `int`), and *variable* is the name of the variable (such as **x** or **myName**). The **equal sign** is used to assign values to the variable.

To create a variable that should store a number, look at the following example:

Example

Create a variable called **myNum** of type `int` and assign it the value **15**:

```
int myNum =15;
cout << myNum;
```

Run example »

You can also declare a variable without assigning the value, and assign the value later:

Example

```
int myNum;
myNum =15;
cout << myNum;
```

Run example »

Note that if you assign a new value to an existing variable, it will overwrite the previous value:

Example

```
int myNum = 15;  // myNum is 15 myNum = 10;  // Now myNum is 10 cout << myNum;  // Outputs 10
```

Run example »

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Other Types

A demonstration of other data types:

Example

```
int myNum =5;              // Integer (whole number without decimals)
doublemyFloatNum =5.99;    // Floating point number (with decimals)
char myLetter ='D';        // Character
string myText ="Hello";    // String (text)
bool myBoolean =true;      // Boolean (true or false)
```

You will learn more about the individual types in the Data Type s chapter.

Display Variables

The `cout` object is used together with the `<<` operator to display variables.

To combine both text and a variable, separate them with the `<<` operator:

Example

```
int myAge =35;
cout <<"I am "<< myAge<< " years old;"
```

Run example »

Add Variables Together

To add a variable to another variable, you can use the `+` operator:

Example

```
int x =
5; int y
= 6; int
sum = x
+ y;
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

```
cout << sum;
```

C++ Data Types

As explained in the Variables chapter, a variable in C++ must be a specified data type:

Example

```
int myNum = 5;               // Integer (whole number)
float myFloatNum = 5.99;     // Floating point number
double myDoubleNum = 9.98;   // Floating point number
char myLetter = 'D';         // Character
bool myBoolean = true;       // Boolean
string myText = "Hello";     // String
```

Run example »

Basic Data Types

The data type specifies the size and type of information the variable will store:

Data Type	Size	Description
int	4 bytes	Stores whole numbers, without decimals
float	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 7 decimal digits
double	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits
boolean	1 byte	Stores true or false values
char	1 byte	Stores a single character/letter/number, or ASCII values

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

You will learn more about the individual data types in the next chapters.

C++ Exercises

Exercise:

Add the correct data type for the following variables:

```
        myNum = 9;
        myDoubleNum = 8.99;
        myLetter = 'A';
        myBool = false;
        myText = "Hello World";
```

C++ Operators

Operators are used to perform operations on variables and values.

In the example below, we use the `+` **operator** to add together two values:

Example

```
int x = 100 + 50;
```

Run example »

Although the `+` operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable:

Example

```
int sum1 = 100 + 50;        // 150
(100 + 50) int sum2 = sum1 + 250;
// 400 (150 + 250) int sum3 = sum2 +
sum2;     // 800 (400 + 400)
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Run example »

C++ divides the operators into the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

Operator	Name	Description	Example	Try it
+	Addition	Adds together two values	x + y	Try it »
-	Subtraction	Subtracts one value from another	x - y	Try it »
*	Multiplication	Multiplies two values	x * y	Try it »
/	Division	Divides one value by another	x / y	Try it »
%	Modulus	Returns the division remainder	x % y	Try it »
++	Increment	Increases the value of a variable by 1	++x	Try it »
--	Decrement	Decreases the value of a variable by 1	--x	Try it »

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

C++ Exercises

Exercise:

Multiply `10` with `5`, and print the result.

```
cout << 10 [ ] 5;
```

C++ Conditions and If Statements

C++ supports the usual logical conditions from mathematics:

- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`
- Equal to `a == b`
- Not Equal to: `a != b`

You can use these conditions to perform different actions for different decisions.

C++ has the following conditional statements:

- Use `if` to specify a block of code to be executed, if a specified condition is true
- Use `else` to specify a block of code to be executed, if the same condition is false
- Use `else if` to specify a new condition to test, if the first condition is false
- Use `switch` to specify many alternative blocks of code to be executed

The if Statement

Use the `if` statement to specify a block of C++ code to be executed if a condition is `true`.

Syntax

```
if (condition) {
  // block of code to be executed if the condition is true
}
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Note that `if` is in lowercase letters. Uppercase letters (If or IF) will generate an error.

In the example below, we test two values to find out if 20 is greater than 18. If the condition is `true`, print some text:

Example

```
if (20 > 18) {
  cout <<"20 is greater than 18"
}
```

Run example »

We can also test variables:

Example

```
int x = 20;
int y = 18;
if (x > y){
  cout <<"x is greater than y"
}
```

Run example »

Example explained

In the example above we use two variables, `x` and `y`, to test whether x is greater than y (using the `>` operator). As x is 20, and y is 18, and we know that 20 is greater than 18, we print to the screen that "x is greater than y".

C++ Switch Statements

Use the `switch` statement to select one of many code blocks to be executed.

Syntax

```
switch(expression) {
  case x:
    //
code
block
break;
case y:
// code
block
break;
default:
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

```
    // code block
}
```

This is how it works:

- The `switch` expression is evaluated once
- The value of the expression is compared with the values of each `case`
- If th ere is a match, the associated block of code is executed
- The `break` and `default` keywords are optional, and will be described later in this chapter

The example below uses the weekday number to calculate the weekday name:

Example

```
int day =4;
switch(day){
  case 1:
    cout <<"Monday";
    break;
  case 2:
    cout <<"Tuesday";
    break;
  case 3:
    cout <<"Wednesday";
    break;
  case 4:
    cout <<"Thursday";
    break;
  case 5:
    cout <<"Friday";
    break;
  case 6:
    cout <<"Saturday";
    break;
  case 7:
    cout <<"Sunday";
    break;
}
// Outputs "Thursday" (day 4)
```

Run example »

The break Keyword

When C++ reaches a `break` keyword, it breaks out of the switch block.

This will stop the execution of more code and case testing inside the block.

When a match is found, and the job is done, it's time for a break. There is no need for more

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

testing.

A break can save a lot of execution time because it "ignores" the execution of all the rest of t he code in the switch block.

The default Keyword

The `default` keyword specifies some code to run if there is no case match:

Example

```
int day =4;
switch(day){
  case 6:
    cout <<"Today is Saturday"
    break;
  case 7:
    cout <<"Today is Sunday"
    break;
  default
    cout <<"Looking forward to the Weekend"
}
// Outputs "Looking forward to the Weekend"
```

C++ Loops

Loops can execute a block of code as long as a specified condition is reached.

Loops are handy because they save time, reduce errors, and they make code more readable.

C++ While Loop

The `while` loop loops through a block of code as long as a specified condition is `true`:

Syntax

```
while (condition) {
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

```
  // code block to be executed
}
```

In the example below, the code in the loop will run over and over again, as long as a variable (i) is less than 5:

Example

```
int i = 0;
while (i < 5) {
  cout << i << "\n";
  i++;
}
```

C++ For Loop

When you know exactly how many times you want to loop through a block of code, use the `for` loop instead of a `while` loop:

Syntax

```
for (statement 1; statement 2; statement 3) {
  // code block to be executed
}
```

Statement 1 is executed (one time) before the execution of the code block.

Statement 2 defines the condition for executing the code block.

Statement 3 is executed (every time) after the code block has been executed.

The example below will print the numbers 0 to 4:

Example

```
for (int i = 0; i < 5; i++) {
  cout << i << "\n";
}
```

Run example »

Example explained

Statement 1 sets a variable before the loop starts (int i = 0).

Statement 2 defines the condition for the loop to run (i must be less than 5). If the condition is true, the loop will start over again, if it is false, the loop will end.

Statement 3 increases a value (i++) each time the code block in the loop has been executed.

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Another Example

This example will only print even values between 0 and 10:

Example

```
for (int i = 0; i <=10; i = i +2) {
  cout << i <<"\n";
}
```

C++ Break

You have already seen the `break` statement used in an earlier chapter of this tutorial. It was used to "jump out" of a `switch` statement.

The `break` statement can also be used to jump out of a **loop**

This example jumps out of the loop when `i` is equal to 4:

Example

```
for (int i = 0; i <10; i++) {
  if (i == 4) {
    break;
  }
  cout << i <<"\n";
}
```

Run example »

C++ Continue

The `continue` statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

This example skips the value of 4:

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Example

```
for (int i = 0; i < 10; i++) {
  if (i == 4) {
    continue;
  }
  cout << i << "\n";
}
```

C++ Arrays

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

To declare an array, define the variable type, specify the name of the array followed by **square brackets** and specify the number of elements it should store:

```
string cars[4];
```

We have now declared a variable that holds an array of four strings. To insert values to it, we can use an array literal - place the values in a comma-separated list, inside curly braces:

```
string cars[4] = {"Volvo", "BMW", "Ford", "Mazda"};
```

To create an array of three integers, you could write:

```
int myNum[3] = {10, 20, 30};
```

Access the Elements of an Array

You access an array element by referring to the index number.

This statement accesses the value of the **first element** in **cars**:

Example

```
string cars[4] = {"Volvo", "BMW",
"Ford", "Mazda"}; cout << cars[0]; //
Outputs Volvo
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Run example »

Note: Array indexes start with 0: [0] is the first element. [1] is the second element, etc.

Change an Array Element

To change the value of a specific element, refer to the index number:

Example

```
cars[0] = "Opel";
```

Example

```
string cars[4] = {"Volvo", "BMW", "Ford", "Mazda"};
cars[0] = "Opel";
cout << cars[0];
// Now outputs Opel instead of Volvo
```

Run example »

Creating Pointers

You learned from the previous chapter, that we can get the **memory address** of a variable by using the & operator:

Example

```
string food = "Pizza"; // A food variable of type string

cout << food;  // Outputs the value of food
(Pizza) cout << &food; // Outputs the memory
address of food (0x6dfed4)
```

Run example »

A **pointer** however, is a variable that **stores the memory address as its value**.

A pointer variable points to a data type (like `int` or `string`) of the same type, and is created with the `*` operator. The address of the variable you're working with is assigned to the pointer:

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Example

```
string food = "Pizza";  // A food variable of type string
string* ptr = &food;    // A pointer variable, with the name ptr, that stores the address of food

// Output the value of food (Pizza)
cout << food << "\n";

// Output the memory address of food (0x6dfed4)
cout << &food << "\n";

// Output the memory address of food with the pointer (0x6dfed4)
cout << ptr << "\n";
```

Run example »

Example explained

Create a pointer variable with the name `ptr`, that **points to** a `string` variable, by using the asterisk sign `*` (`string* ptr`). Note that the type of the pointer has to match the type of the variable you're working with.

Use the `&` operator to store the memory address of the variable called `food`, and assign it to the pointer.

Now, `ptr` holds the value of `food`'s memory address.

Tip: There are three ways to declare pointer variables, but the first way is preferred:

```
string* mystring; // Preferred
string *mystring;
string * mystring;
```

C++ Exercises

Test Yourself With Exercises

Exercise:

Create a **pointer** variable with the name `ptr`, that should point to a `string` variable named `food`:

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

```
string food = "Pizza";
            = &      ;
```

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.

Create a Function

C++ provides some pre-defined functions, such as `main()`, which is used to execute code. But you can also create your own functions to perform certain actions.

To create (often referred to as *declare*) a function, specify the name of the function, followed by parentheses `()`:

Syntax

```
void myFunction() {
  // code to be executed
}
```

Example Explained

- `myFunction()` is the name of the function
- `void` means that the function does not have a return value. You will learn more about return values later in the next chapter
- inside the function (the body), add code that defines what the function should do

---

Call a Function

Declared functions are not executed immediately. They are "saved for later use", and will be executed later, when they are called.

To call a function, write the function's name followed by two parentheses `()` and a semicolon `;` In the following example, `myFunction()` is used to print a text (the action), when it is called:

Example

Inside `main`, call `myFunction()`:

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

```
// Create a function void myFunction()
{   cout << "I just got executed!";
}
int main() {   myFunction(); // call
the function   return 0;
}

// Outputs "I just got executed!"
```

Run example »

Example

e called multiple times:

```
void myFuncti
{   cout <<
}
int main() {
myFunction();just got executed!";
myFunction()
myFunction();
return 0;
}            ;

// I just got
executed!
// I just got
executed!
// I just got
executed!
```

Run example »

Function Overloading

With **function overloading**, multiple functions can have the same name with different parameters:

Example

```
int myFunction(int x)
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

```
float myFunction(float x)
double myFunction(double x, double y)
```

Consider the following example, which have two functions that add numbers of different type:

Example

```
int plusFuncInt(int x, int y) {
  return x + y;
}

double plusFuncDouble(double x, double y) {
  return x + y;
}

int main() {
  int myNum1 = plusFuncInt(8, 5);
  double myNum2 = plusFuncDouble(4.3, 6.26);
  cout << "Int: " << myNum1 << "\n";
  cout << "Double: " << myNum2;
  return 0;
}
```

Run example »

Instead of defining two functions that should do the same thing, it is better to overload one.

In the example below, we overload the `plusFunc` function to work for both `int` and `double`:

Example

```
int plusFunc(int x, int y) {
  return x + y;
}

double plusFunc(double x, double
y) {   return x + y;
}  int main() {
int myNum1 =
plusFunc(8, 5);
double myNum2 =
plusFunc(4.3, 6.26);
cout << "Int: " <<
myNum1 << "\n";
cout << "Double: "
<< myNum2;   return
0; }
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

C++ What is OOP?

OOP stands for Object -Oriented Programming.

Procedural programming is about writing procedures or functions that perform operations on the data, while object -oriented programming is about creating objects that contain both data and functions.

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C++ code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

Tip: The "Don't Repeat Yourself" (DRY) principle is about reducing the repetition of code. You should extract out the codes that are common for the application, and place them at a single place and reuse them instead of repeat ing it.

C++ What are Classes and Objects?

Classes and objects are the two main aspects of object -oriented programming.

Look at the following illustration to see the difference between class and objects:

class

Fruit

objects

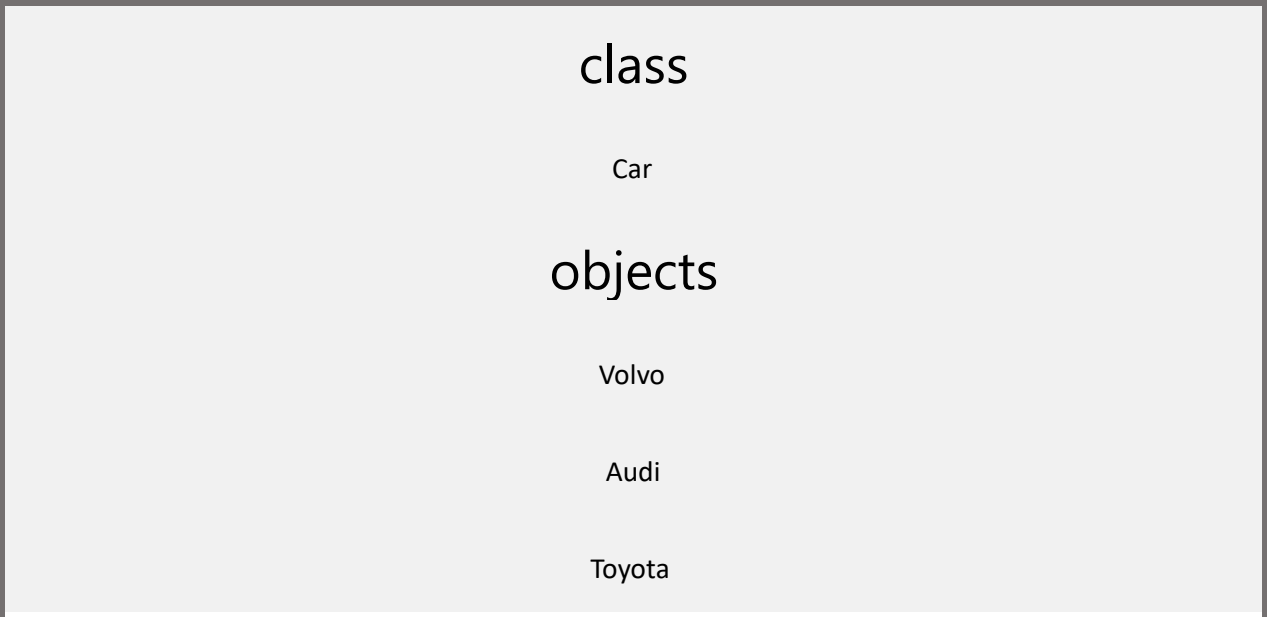
Apple

Banana

Mango

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Another example:



So, a class is a template for objects, and an object is an instance of a class.

When the individual objects are created, they inherit all the variables and functions from the class.

You will learn much more about classes and objects in the next chapter.

C++ Classes/Objects

C++ is an object-oriented programming language.

Everything in C++ is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an **object**. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.

Attributes and methods are basically **variables** and **functions** that belongs to the class. These are often referred to as "class members".

A class is a user-defined data type that we can use in our program, and it works as an object constructor, or a "blueprint" for creating objects.

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Create a Class

To create a class, use the `class` keyword:

Example

Crea te a class called " `MyClass`":

```
classMyClass {      // The class
  public            // Access specifier
    int myNum;      // Attribute (int variable)
    string myString;// Attribute (string variable)
};
```

Example explained

- The `class` keyword is used to create a class called `MyClass`.
- The `public` keyword is an **access specifier** which specifies that members (attributes and methods) of the class are accessible from outside the class. You will learn more about access specifier s later
- Inside the class, there is an integer variable `myNum` and a string variable `myString`. When variables are declared within a class, they are called **attributes**
- At last, end the class definition with a semicolon `;`.

Create an Object

In C++, an object is created from a class. We have already created the class named `MyClass`, so now we can use this to create objects.

To create an object of `MyClass`, specify the class nam e, followed by the object name.

To access the class attributes (`myNum` and `myString`), use the dot syntax (`.`) on the object :

Example

Create an object called " `myObj`" and access the attributes:

```
classMyClass {      // The class
  public            // Accessspecifier
    int myNum;        // Attribute (int
variable)     string myString;  //
Attribute (string variable)
};

int main() {
  MyClass myObj;  // Create an object of MyClass
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

```
  // Access attributes and
set values   myObj.myNum =
15;    myObj.myString =
"Some text";

  // Print attribute values
  cout << myObj.myNum << "\n";
  cout << myObj.myString;

}
```

Run example »

Multiple Objects

You can create multiple objects of one class:

Example

```
// Create a Car class with some attributes
class Car {
  public:
    string brand;
    string model;
    int year;
};

int main() {
  // Create an object of Car
  Car carObj1;
  carObj1.brand = "BMW";
  carObj1.model = "X5";
  carObj1.year = 1999;

  // Create another object of Car
  Car carObj2;
  carObj2.brand = "Ford";
return 0;
  carObj2.model = "Mustang";
carObj2.year = 1969;

  // Print attribute values   cout << carObj1.brand << " " <<
carObj1.model << " " << carObj1.year << "\n";   cout <<
carObj2.brand << " " << carObj2.model << " " << carObj2.year <<
"\n";   return 0; }
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Constructors

A constructor in C++ is a **special method** that is automatically called when an object of a class is created.

To create a constructor, use the same name as the class, followed by parentheses `()`:

Example

```
class MyClass {     // The class
  public:           // Access specifier
    MyClass() {     // Constructor
      cout << "Hello World!";
    }
};

int main() {
  MyClass myObj;    // Create an object of MyClass (this will call the constructor)
  return 0;
}
```

Run example »

Note: The constructor has the same name as the class, it is always `public`, and it does not have any return value.

Constructor Parameters

Constructors can also take parameters (just like regular functions), which can be useful for setting initial values for attributes.

The following class have `brand`, `model` and `year` attributes, and a constructor with different parameters. Inside the constructor we set the attributes equal to the constructor parameters (`brand=x`, etc). When we call the constructor (by creating an object of the class), we pass parameters to the constructor, which will set the value of the corresponding attributes to the same:

Example

```
class Car {         // The class    public:           // Access specifier
string brand;  // Attribute
    string model;  //
Attribute     int year;
// Attribute
    Car(string x, string y, int z) { // Constructor
with parameters       brand = x;       model = y;
year = z;
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

```
    }
};

int main() {
  // Create Car objects and call the constructor with different values
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);

  // Print values
  cout << carObj1.brand << " " << carObj1.model << " " << carObj1.year << "\n";
  cout << carObj2.brand << " " << carObj2.model << " " << carObj2.year << "\n";
  return 0;
}
```

Run example »

Just like functions, constructors can also be defined outside the class. First, declare the constructor inside the class, and then define it outside of the class by specifying the name of the class, followed by the scope resolution :: operator, followed by the name of the constructor (which is the same as the class):

Example

```
class Car {       // The class
  public:         // Access specifier
    string brand;// Attribute
    string model;// Attribute
    int year;     // Attribute
    Car(string x, string y, int z); // Constructor declaration
};

// Constructor definition outside the class
Car::Car(string x, string y, int z) {
  brand = x;
  model = y;
  year = z;
}
 int
main(
) {
  // Create Car objects and call the constructor with different values
  Car carObj1("BMW", "X5", 1999);
  Car carObj2("Ford", "Mustang", 1969);

  // Print values   cout << carObj1.brand << " " <<
carObj1.model << " " << carObj1.year << "\n";   cout <<
carObj2.brand << " " << carObj2.model << " " << carObj2.year <<
"\n";   return 0;
}
```

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM

Run example »

Inheritance

In C++ it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:

- **derived class** (child) - the class that inherits from another class
- **base class** (parent) - the class being inherited from

To inherit from a class, use the `:` symbol.

In the example below, the `Car` class (child) inherits the attributes and methods from the `Vehicle` class (parent):

Example

```
class Base class
      Vehicle {
    ublic:
                brand ="Ford";
     void cout <nk() {
};     }
                  "Tuut, tuut\n" ;

class
     Derived cla
};   ar: public:
int                publicVehicle{
                   model ="Mustang";


     Car myCar;
     ain() {
  myCar.honk();   cout <<
myCar.brand + " " +
myCar.model;   return 0;
}
```

www.vuan...

Regards: Vu-Gateway

Click: https://chat.whatsapp.com/GecIrau2Nit0D4F5DjDiGM